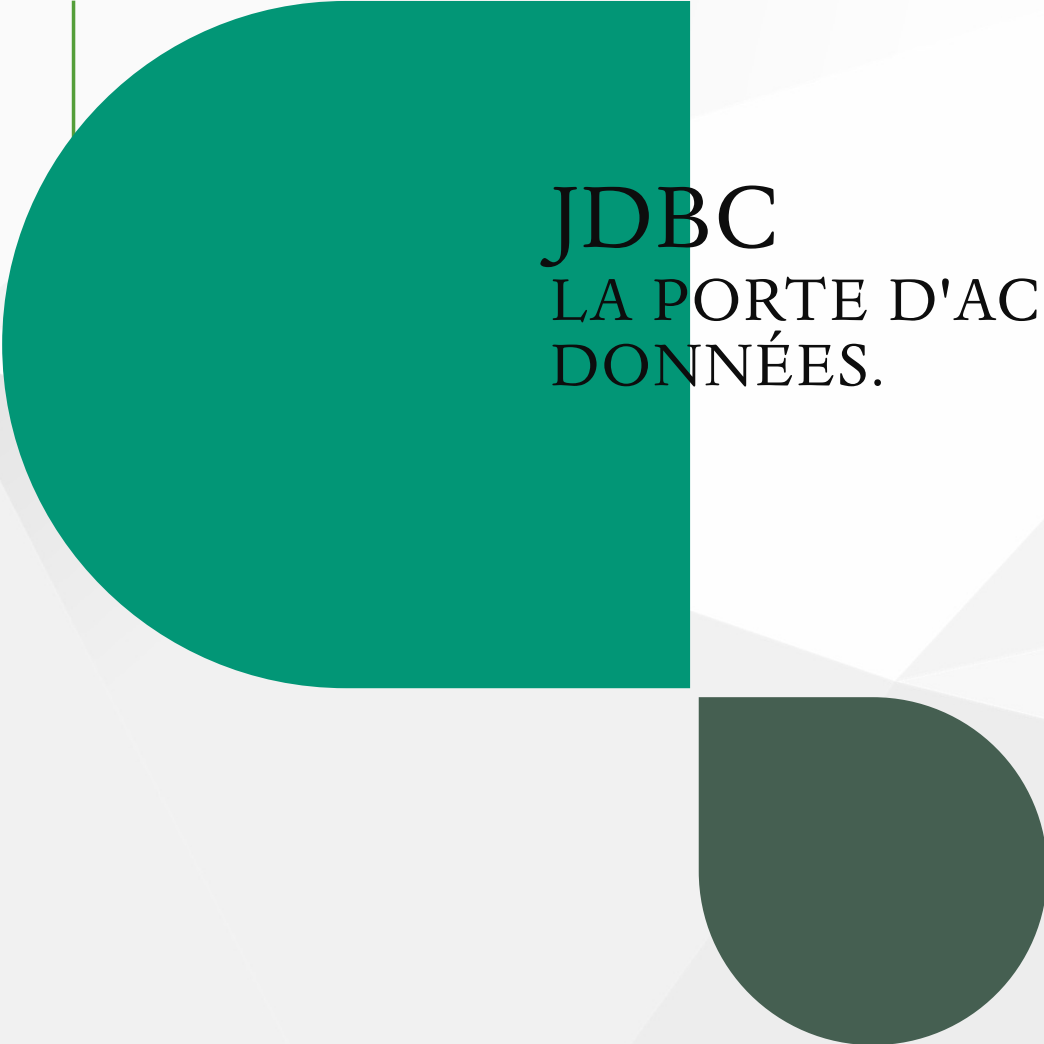




Java

JDBC et La porte d'accès aux bases de données.





JDBC

LA PORTE D'ACCÈS AUX BASES DE
DONNÉES.

ASTUCE JAVA-BDD

Dans le cadre de nos bases, une bonne pratique est d'utiliser les [Wrapper](#) au lieu des types primitif.

1. Primitif `long number;`
2. [Wrapper](#) `Long number;`

Pourquoi ? Un [Wrapper](#) est un objet puisqu'il est défini par une classe. De ce fait, l'objet étant capable d'accepter la valeur null, vous pouvez facilement récupérer toutes les valeurs null provenant de la BDD.

Enfin, un [Wrapper](#) vous permet d'avoir accès à des méthodes pour manipuler le type, ce qui est confortable dans nos différents développements.



EN L'OCCURRENCE, LES CHAMPS DE NOTRE TABLE SQL SE VOIENT TOUTES APPLIQUER UNE CONTRAINTE NULL, IL Y A DONC RISQUE DE VALEURS NULLES ET GÉNÉRATION D'EXCEPTIONS



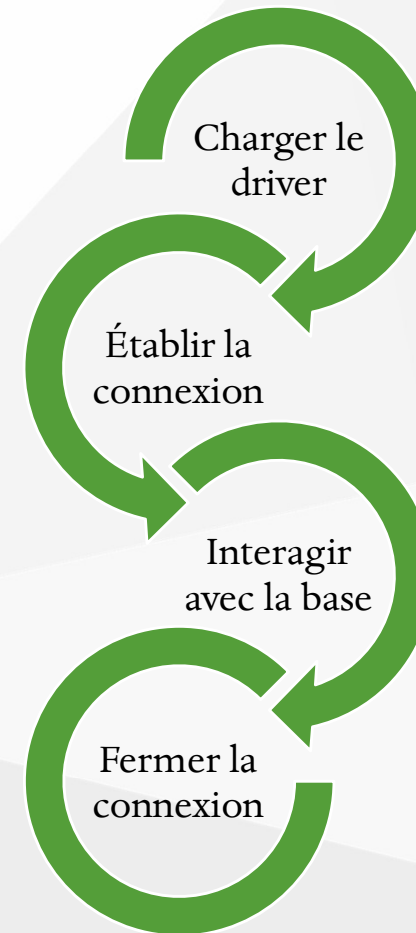
CEPENDANT, C'EST UNE BONNE PRATIQUE DE TOUJOURS UTILISER LES OBJETS WRAPPER DANS LES BEANS DONT LES PROPRIÉTÉS CORRESPONDENT À DES CHAMPS D'UNE BASE DE DONNÉES, AFIN D'ÉVITER CE GENRE D'ERREURS !

ETAPE POUR CRÉER UNE CONNEXION

Pour créer une connexion à une base de données avec le langage JAVA, il y a plusieurs étapes successives à réaliser.

1. Charger le driver de la BDD
2. Établir la connexion à la BDD
3. Faire les différentes interactions (CRUD)
4. Fermer la connexion.

Comme nous faisons appel à une ou des ressource(s), il faut également penser à capturer toutes les exceptions provenant de la BDD.



INSTALLATION

Télécharger l'extension

- Pour pouvoir connecter votre application Java à une base de données MySQL, il vous faudra télécharger l'extension (driver) "MySQL Connector.jar"

<https://dev.mysql.com/downloads/connector/j/>

Chaque SGBD a son driver donc par conséquent, il faut penser à correctement choisir le bon driver.

- Pour MySQL, vous pouvez trouver le nom du driver à utiliser sur <https://dev.mysql.com/doc/connector-j/en/connector-j-reference-driver-name.html>

Installer de l'extension

Dans notre JDK, nous avons l'ensemble des classes pour interagir avec la base de données.

L'extension "MySQL Connector" ne fait pas partie de la JDK.

Il faut donc ajouter cette librairie à notre projet. Pour cela, il existe 2 manières de faire :

1. Soit l'ajouter à la main.
2. Soit passer par un gestionnaire de projet (ex: Maven).

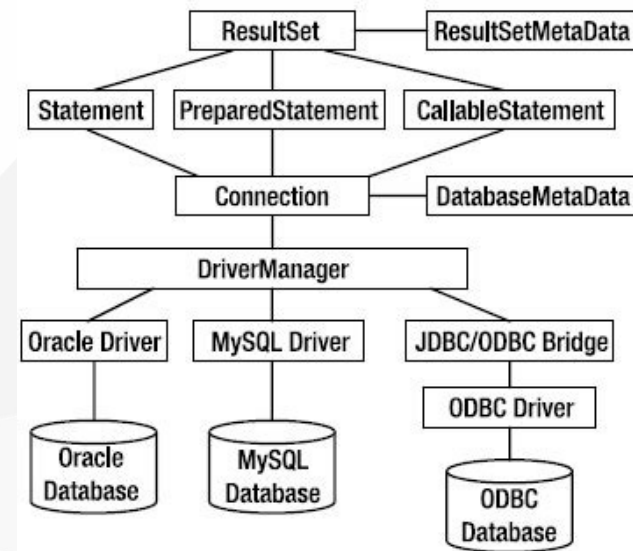
JAVA DATABASE CONNECTIVITY (JDK)

Il s'agit de classes Java contenu dans le package `Java.sql` permettant de se connecter et d'interagir avec des bases de données.

- La base de données est prête, les tables sont créées, remplies et nous avons le `driver` nécessaire à notre base de données.

On crée donc une instance de l'objet `Driver` permettant d'ouvrir une `Connection`, permettant d'obtenir un `Statement` qui nous retournera un `ResultSet`

Enfin, pour terminer proprement une connexion, il faut fermer les différents espaces ouverts. Chaque objet possède une méthode `close()`





CRÉATION DE LA CONNEXION

Se connecter à la BDD

1ÈRE CONNEXION

Pour effectuer une connexion à la base de données, nous avons besoin d'une **url**, d'un **login** et d'un **mot de passe**.

- **L'url** : protocol//[hosts][/database][?properties]
- **Login** : compte de connexion
- **Mot de passe** : mot de passe de connexion

La plupart des méthodes lèvent l'exception `java.sql.SQLException` qui étend la classe `Exception`.

```
private void firstConnection() { 1 usage new *
    String BDD = "airdejawa";
    String url = "jdbc:mysql://localhost:3306/airdejawa";
    String user = "root";
    String password = "root";

    try {
        // chargement du driver
        Class.forName( "className: com.mysql.cj.jdbc.Driver");

        // Création de la connexion
        Connection connection = DriverManager.getConnection(url, user, password);

        // petit message indiquant que tout s'est bien passé
        System.out.println("Connected to database");

        // fermeture de la connexion
        connection.close();

    } catch (ClassNotFoundException e) {
        // exception pour le driver
        throw new RuntimeException(e);
    } catch (SQLException e) {
        // exception pour SQL
        throw new RuntimeException(e);
    }
}
```


FICHIER DE CONFIGURATION

Pour l'exemple, l'utilisation de variables placées dans le code n'est pas conseillée.

Il est préférable d'utiliser des fichiers de configurations qu'on place dans notre architecture et qu'on va ensuite lire pour en déduire les données de configuration.

Puisque ce fichier contient des informations confidentielles, comme le mot de passe de connexion à la base de données, il est hors de question de le placer dans un répertoire public de l'application.

On va alors créer un répertoire dans notre application en dehors du dossier src contenant notre code.

Dans une vraie application, les fichiers de configuration ne sont pas stockés au sein de l'application, mais en dehors. Il devient ainsi très facile d'y modifier une valeur sans avoir à naviguer dans le contenu de l'application.

```
1#conf DB
2jdbc.driver.class=com.mysql.cj.jdbc.Driver
3jdbc.url=jdbc:mysql://localhost:3306/airdejava
4jdbc.login=root
5jdbc.password=root
```

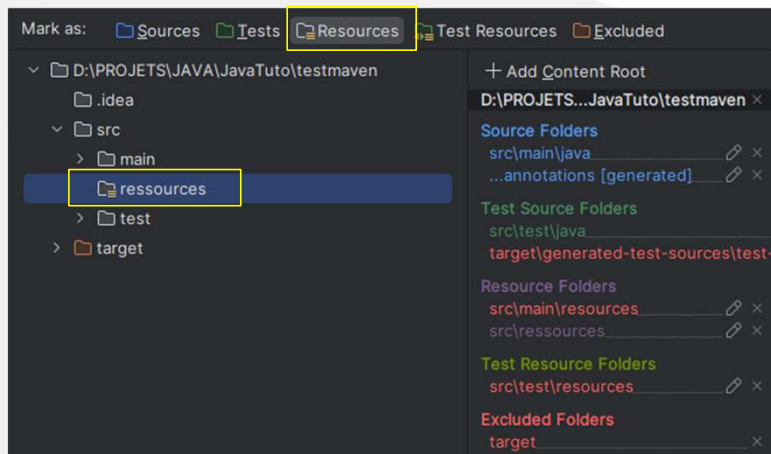
Exemple de fichier conf.properties

Il fonctionne selon le principe clé=valeur

UTILISATION DU FICHIER DE CONFIGURATION

Votre fichier de configuration va être placé dans un repertoire qu'on va déclarer dans notre application comme ressources.

Pour cela, avec IntelliJ, il suffit d'aller dans [open module settings](#) et de déclarer votre dossier comme ressources (un petit symbole apparaît sur le dossier).



```
private void propertiesConnection() { usage new *

    // chemin vers le fichier de proprietes
    final String PATHCONF = "conf.properties";
    // attribut proprietes
    Properties prop = new Properties();

    // chargement du fichier de proprietes
    try (InputStream is = getClass().getClassLoader().getResourceAsStream(PATHCONF)) {

        prop.load(is);

    } catch (IOException e) {
        throw new RuntimeException(e);
    }

    // traitement du fichier de proprietes et de la connexion a la BDD
    try {

        // chargement du driver
        Class.forName( prop.getProperty("jdbc.driver.class"));

        // recuperation des parametres pour la connexion
        String url = prop.getProperty("jdbc.url");
        String user = prop.getProperty("jdbc.login");
        String password = prop.getProperty("jdbc.password");

        Connection connection = DriverManager.getConnection(url, user, password);

        // petit message indiquant que tout s'est bien passé
        System.out.println("Connected to database from properties");

        // fermeture de la connexion
        connection.close();

    } catch (ClassNotFoundException e) {
        throw new RuntimeException(e);
    } catch (SQLException e) {
        throw new RuntimeException(e);
    }

}
```



INTERACTION

Utiliser la BDD

MANIPULATION DE COMMANDE SQL

- La requête est créée avec l'interface `Statement` qui possède les méthodes nécessaires pour réaliser les requêtes sur la base associée à la connexion.
- 3 types (interfaces) d'objets statement :
 - `Statement` : requêtes simples (SDSL Statique)
 - `PreparedStatement` : requêtes paramétrables (ou dynamiques) précompilées (requêtes avec paramètres d'entrée/sortie)
 - `CallableStatement` : encapsule procédures SQL stockées dans le SGBD
- Création d'un statement :
 - `Statement stm = connexion.createStatement();`
 - `PreparedStatement stm = connexion.prepareStatement(str);`
 - `CallableStatement stm = connexion.prepareCall(str);`

```
// un simple select
private void selectFromPersonne(Connection con) { 1 usage new *

    String selectSQL = "select * from personne";

    try {
        // etablisement d'un Statement
        Statement stmt = con.createStatement();

        // utilisation du statement pour envoyer la requête et récupération du résultat
        ResultSet resultSet = stmt.executeQuery(selectSQL);

        while(resultSet.next()) {
            System.out.println(resultSet.getString( columnLabel: "per_nom"));
            System.out.println(resultSet.getString( columnLabel: "per_prenom"));
            System.out.println(resultSet.getString( columnLabel: "per_adr"));
            System.out.println(resultSet.getString( columnLabel: "per_cp"));
            System.out.println(resultSet.getString( columnLabel: "per_ville"));
            System.out.println(resultSet.getString( columnLabel: "per_tel"));
            System.out.println(resultSet.getString( columnLabel: "per_email"));
            System.out.println("-----");
        }
    } catch (SQLException e) {
        throw new RuntimeException(e);
    }
}
```

```
private List<Personne> findAll(Connection con) { 1 usage new *

    List<Personne> personnes = new ArrayList<>();
    String selectSQL = "select * from personne";

    try {
        // etablisement d'un Statement
        Statement stmt = con.createStatement();

        // utilisation du statement pour envoyer la requête et récupération du résultat
        ResultSet resultSet = stmt.executeQuery(selectSQL);

        while(resultSet.next()) {

            Personne personne = new Personne();

            personne.setId(resultSet.getInt( columnLabel: "per_id"));
            personne.setCivilditeId(resultSet.getInt( columnLabel: "civ_id"));
            personne.setNom(resultSet.getString( columnLabel: "per_nom"));
            personne.setPrenom(resultSet.getString( columnLabel: "per_prenom"));
            personne.setAdresse(resultSet.getString( columnLabel: "per_adr"));
            personne.setEmail(resultSet.getString( columnLabel: "per_cp"));
            personne.setVille(resultSet.getString( columnLabel: "per_ville"));
            personne.setTelephone(resultSet.getInt( columnLabel: "per_tel"));
            personne.setFax(resultSet.getInt( columnLabel: "per_fax"));
            personne.setEmail(resultSet.getString( columnLabel: "per_email"));
            personne.setDateNaissance(resultSet.getDate( columnLabel: "per_datenaissance"));

            personnes.add(personne);
        }

        return personnes;
    } catch (SQLException e) {
        throw new RuntimeException(e);
    }
}
```

UTILISATION DE L'OBJET

Comme désormais, on maîtrise l'objet, la bonne manière de faire est de créer un objet métier.

UTILISATION DE STRINGBUILDER

Lorsqu'on vient à créer nos requêtes, il est préférable d'utiliser la classe `StringBuilder` à la place de la classe `String`.

Pourquoi ? Nos requêtes vont être une association de chaîne de caractères par concaténation le plus souvent.

Pour rappel, `String` est immuable c'est-à-dire que toutes modifications apportées sur `String` produisent un autre objet `String`.

`StringBuilder` ou `StringBuffer` sont eux muables.

```
StringBuilder sqlFindAclient = new StringBuilder();
sqlFindAclient.append("select ");
sqlFindAclient.append("c.idsociety,");
sqlFindAclient.append("socialreason,");
sqlFindAclient.append("domainsoc,");
sqlFindAclient.append("numstreet,");
sqlFindAclient.append("namestreet,");
sqlFindAclient.append("postcode,");
sqlFindAclient.append("city,");
sqlFindAclient.append("phone,");
sqlFindAclient.append("email,");
sqlFindAclient.append("commentsoc,");
sqlFindAclient.append("revenue,");
sqlFindAclient.append("workforce ");
sqlFindAclient.append("from \"client\" c ");
sqlFindAclient.append("join \"society\" s ");
sqlFindAclient.append("on c.idsociety = s.idsociety ");
sqlFindAclient.append("where c.idsociety = ?");
```


L'INTERFACE RESULTSET

Ensuite ce **Statement** offre des méthodes permettant de passer des ordre SQL à exécuter.

3 méthodes d'exécutions d'un Statement :

- **ResultSet executeQuery(String SQL)** : pour les requêtes qui retournent un résultat (SELECT) accessible au travers d'un objet ResultSet
- **Int executeUpdate(String SQL)** : pour les requêtes qui ne retournent pas de résultats (INSERT, DELETE, UPDATE...)
- **boolean execute(String SQL)** : quand on ne sait pas si la requête retourne ou non un résultat, procédures stockées.

```
// utilisation du statement pour envoyer la requête et récupération du
ResultSet resultSet = stmt.executeQuery(selectSQL);

while(resultSet.next()) {

    Personne personne = new Personne();

    personne.setId(resultSet.getInt( columnLabel: "per_id"));
    personne.setCivilliteId(resultSet.getInt( columnLabel: "civ_id"));
    personne.setNom(resultSet.getString( columnLabel: "per_nom"));
    personne.setPrenom(resultSet.getString( columnLabel: "per_prenom"));
    personne.setAdresse(resultSet.getString( columnLabel: "per_adr"));
    personne.setEmail(resultSet.getString( columnLabel: "per_cp"));
    personne.setVille(resultSet.getString( columnLabel: "per_ville"));
    personne.setTelephone(resultSet.getInt( columnLabel: "per_tel"));
    personne.setFax(resultSet.getInt( columnLabel: "per_fax"));
    personne.setEmail(resultSet.getString( columnLabel: "per_email"));
    personne.setDateNaissance(resultSet.getDate( columnLabel: "per_datenai

    personnes.add(personne);
}
```

L'INTERFACE RESULTSET

`executeQuery()` renvoie une instance de `ResultSet` qui permet d'accéder aux champs des n-uplets sélectionnés.

Les rangées du `ResultSet` se parcourent itérativement ligne par ligne.

- `Boolean next()` permet d'avancer à la ligne suivante, retourne false si pas de ligne suivante placé avant la première ligne à la création du `ResultSet`
- Les colonnes sont référencées par leur numéro ou par leur nom
- L'accès aux valeurs des colonnes se fait par des méthodes `getXXX(String nomCol)` ou `getXXX(int numCol)` où XXX dépend du type de la colonne de la table SQL

	TINYINT	SMALLINT	INTEGER	BIGINT	REAL	FLOAT	DOUBLE	DECIMAL	NUMERIC	BIT	CHAR	VARCHAR	LONGVARCHAR	BINARY	VARBINARY	LONGVARBINARY	DATE	TIME	TIMESTAMP	CLOB	BLOB	ARRAY	REF	STRUCT	JAVA OBJECT
<code>getBytes</code>	X	x	x	x	x	x	x	x	x	x	x	x	x												
<code>getShort</code>	x	X	x	x	x	x	x	x	x	x	x	x	x												
<code>getInt</code>	x	x	X	x	x	x	x	x	x	x	x	x	x												
<code>getLong</code>	x	x	x	X	x	x	x	x	x	x	x	x	x												
<code>getFloat</code>	x	x	x	x	X	x	x	x	x	x	x	x	x												
<code>getDouble</code>	x	x	x	x	x	X	X	x	x	x	x	x	x												
<code>getBigDecimal</code>	x	x	x	x	x	x	x	X	X	x	x	x	x												
<code>getBoolean</code>	x	x	x	x	x	x	x	x	x	X	x	x	x												
<code>getString</code>	x	x	x	x	x	x	x	x	x	x	X	X	x	x	x	x	x	x							
<code>getBytes</code>														X	X	x									
<code>getDate</code>											x	x	x				X	x							
<code>getTime</code>											x	x	x					X	x						
<code>getTimestamp</code>											x	x	x				x	x	X						
<code>getAsciiStream</code>											x	x	X	x	x	x									
<code>getUnicodeStream</code>											x	x	X	x	x	x									
<code>getBinaryStream</code>														x	x	X									
<code>getClob</code>																			X						
<code>getBlob</code>																				X					
<code>getArray</code>																					X				
<code>getRef</code>																						X			
<code>getCharacterStream</code>											x	x	X	x	x	x									
<code>getObject</code>	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	X	X

Table 5.1: Use of `ResultSet.getXXX` Methods to Retrieve JDBC Types

LES REQUÊTES PRÉPARÉES

Si on doit répéter plusieurs fois la même requête (par exemple `select * from login where log_user = num_id`), alors la requête préparée permet d'écrire une seule requête dans laquelle on prévoit un ou plusieurs paramètres :

- *Select * from login where log_user = ?*

Le symbole `?` désigne un paramètre dont la valeur sera fixée par la suite.

Une requête préparée est transmise au SGBR à l'aide de la méthode `prepareStatement` de la classe `Connection` qui nous renvoie un objet de type `PreparedStatement` contenant toutes les informations nécessaires à l'exploitation de cette requête préparée par le SGBDR.



```
private Personne findById(Connection con, int pId) { 1 usage new *
    Personne personne = new Personne();
    String selectById = "select * from personne where per_id=?";

    try {
        // utilisation d'un preparedStatement
        PreparedStatement pstmt = con.prepareStatement(selectById);

        pstmt.setInt( parameterIndex: 1, pId);

        ResultSet resultSet = pstmt.executeQuery();

        while(resultSet.next()) {

            personne.setId(resultSet.getInt( columnLabel: "per_id"));
            personne.setCivileId(resultSet.getInt( columnLabel: "civ_id"));
            personne.setNom(resultSet.getString( columnLabel: "per_nom"));
            personne.setPrenom(resultSet.getString( columnLabel: "per_prenom"));
            personne.setAdresse(resultSet.getString( columnLabel: "per_adr"));
            personne.setEmail(resultSet.getString( columnLabel: "per_cp"));
            personne.setVille(resultSet.getString( columnLabel: "per_ville"));
            personne.setTelephone(resultSet.getInt( columnLabel: "per_tel"));
            personne.setFax(resultSet.getInt( columnLabel: "per_fax"));
            personne.setEmail(resultSet.getString( columnLabel: "per_email"));
            personne.setDateNaissance(resultSet.getDate( columnLabel: "per_datenaissance"));

        }

        return personne;

    } catch (SQLException e) {
        throw new RuntimeException(e);
    }
}
```

LES REQUÊTES PRÉPARÉES

Ensuite, on peut transmettre au SGBDR, les valeurs voulues des paramètres, à l'aide des méthodes de la forme `setXXX(rang, valeur)` de la classe `PreparedStatement`, auxquelles on précise :

- Le rang du paramètre concerné
- La valeur prévue pour ce paramètre.

Enfin, on exécute la requête par l'une de des méthodes vu ultérieurement.



```
System.out.println("Affichage d'une personne");  
System.out.println(findById(connection, pld: 41));
```

Affichage d'une personne
41 / 2 - BRETANI MARIE - 56 RUE DE LA LORRAINE null MARSEILLE

RÉCUPÉRER UN ID AUTO- GÉNÉRÉ

Avec le `prepareStatement`, on peut récupérer une information sur la requête réalisée.

Par exemple dans le cas d'un insert, si on souhaite récupérer l'ID auto-généré, il faut indiquer lors que la requête préparée, à notre `prepareStatement` le besoin de retourner l'information

<https://docs.oracle.com/javase/8/docs/api/java/sql/Connection.html#prepareStatement-java.lang.String-int->

```
private int insertPersonne(Connection con, Personne personne) { 1 usage new *

    SimpleDateFormat simpleDateFormat = new SimpleDateFormat( pattern: "yyyy-MM-dd");
    int newId = 0;

    StringBuilder insertSQL = new StringBuilder();
    insertSQL.append("insert into Personne (civ_id,per_nom,per_prenom,per_adr,");
    insertSQL.append("per_cp, per_ville, per_datenaissance) ");
    insertSQL.append("values (?,?,?,?,,?)");

    try {
        // preparation de la requete
        PreparedStatement pstmt = con.prepareStatement(insertSQL.toString(),
            PreparedStatement.RETURN_GENERATED_KEYS);

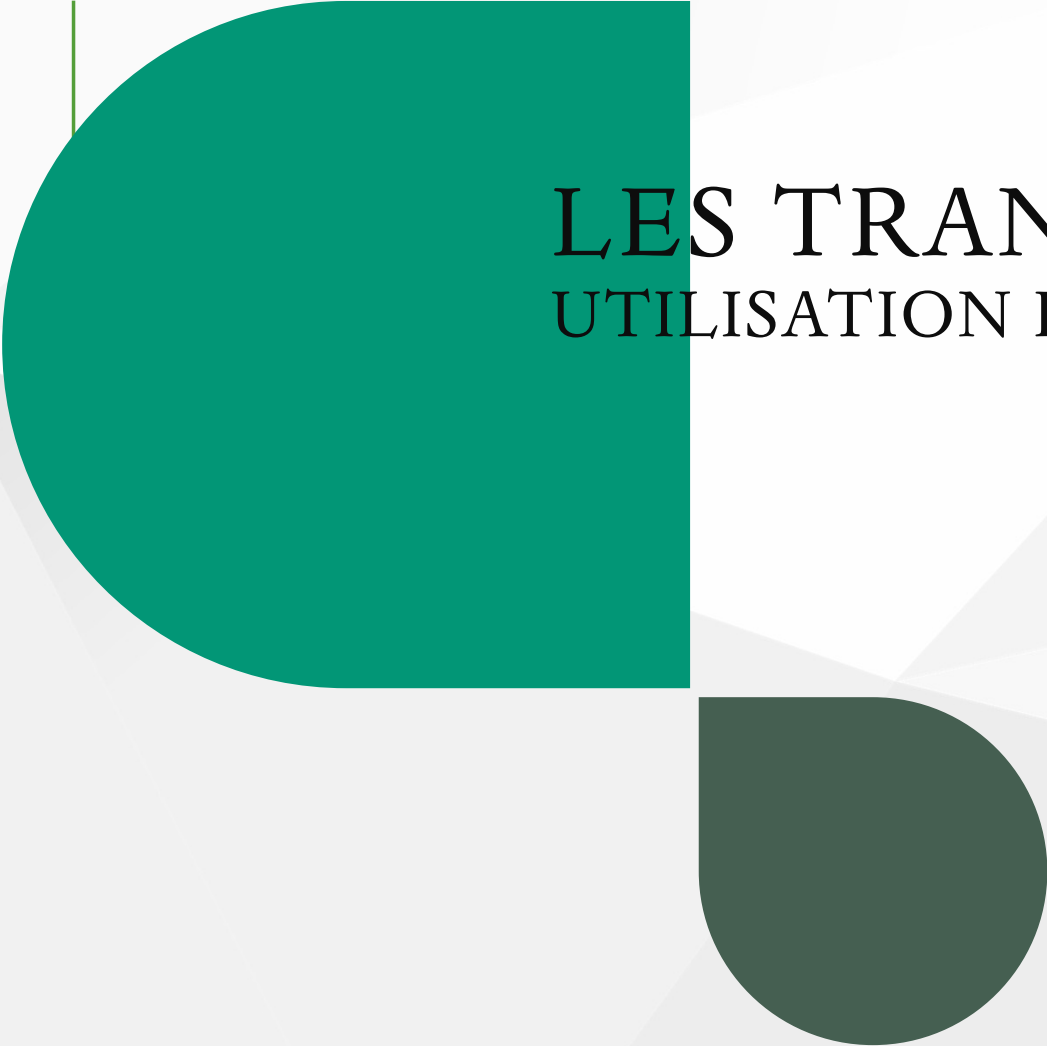
        // mise en place des parametres
        pstmt.setInt( parameterIndex: 1, personne.getCiviliteId());
        pstmt.setString( parameterIndex: 2, personne.getNom());
        pstmt.setString( parameterIndex: 3, personne.getPrenom());
        pstmt.setString( parameterIndex: 4, personne.getAdresse());
        pstmt.setString( parameterIndex: 5, personne.getCodePostal());
        pstmt.setString( parameterIndex: 6, personne.getVille());

        // conversion en SQL DATE
        String formattedDate = simpleDateFormat.format(personne.getDateNaissance());
        pstmt.setDate( parameterIndex: 7, Date.valueOf(formattedDate));

        // execution
        pstmt.executeUpdate();

        // recuperation de l'ID
        ResultSet rs = pstmt.getGeneratedKeys();
        if (rs.next()) {
            newId = rs.getInt( columnIndex: 1);
        }

        return newId;
    } catch (SQLException e) {
        throw new RuntimeException(e);
    }
}
```



LES TRANSACTIONS

UTILISATION EN JAVA

TRANSACTION

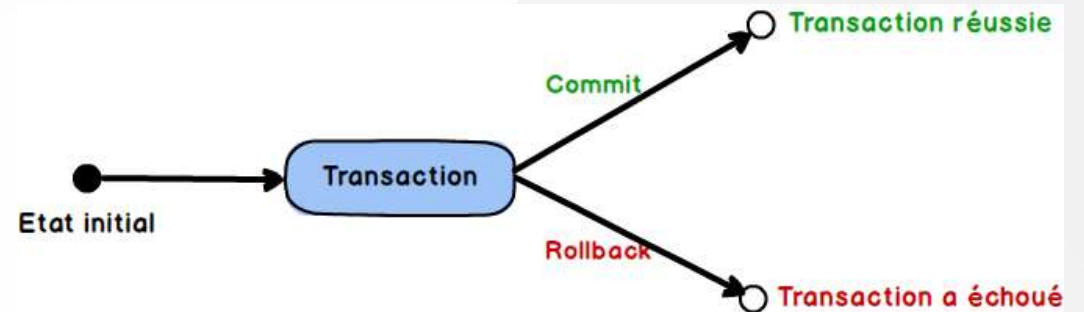
- Par défaut, la connexion est en "auto-commit". Un commit est automatiquement lancé après chaque ordre SQL qui modifie la base.
- Pour définir une transaction composée de plusieurs requêtes SQL, il faut désactiver l'auto-commit :

`connect.setAutoCommit(false);`

- Il faut alors explicitement valider ou annuler la transaction par :

`connect.commit()` : valide la transaction

`connect.rollback()` : annule la transaction



```

try{
    //définir la gestion des transactions manuelles
    conn.setAutoCommit(false);
    Statement stmt = conn.createStatement();

    String query = "INSERT INTO login VALUES (1, 'John', 'tx514')";
    stmt.executeUpdate(query);

    //Soumettre une instruction SQL mal formée qui brise le code
    String query = "INSERT IN login VALUES (2, 'invit', 'rx897')";
    stmt.executeUpdate(query);

    // S'il n'y a pas d'erreur.
    conn.commit();
}
catch(SQLException se){
    // S'il y a une erreur.
    conn.rollback();
}
  
```

LES TRANSACTIONS

Il faut noter que :

- Pas besoin d'effectuer `START TRANSACTION` lors de votre transaction puisque les requêtes sont explicitement encadrées par un `START TRANSACTION` par le SGBD.
- Il est possible de protéger nos transactions pour éviter "des lectures sales" avec un niveau d'isolation qui appliquent des règles d'accès très strictes.
- <https://docs.oracle.com/javase/tutorial/jdbc/basics/transactions.html>

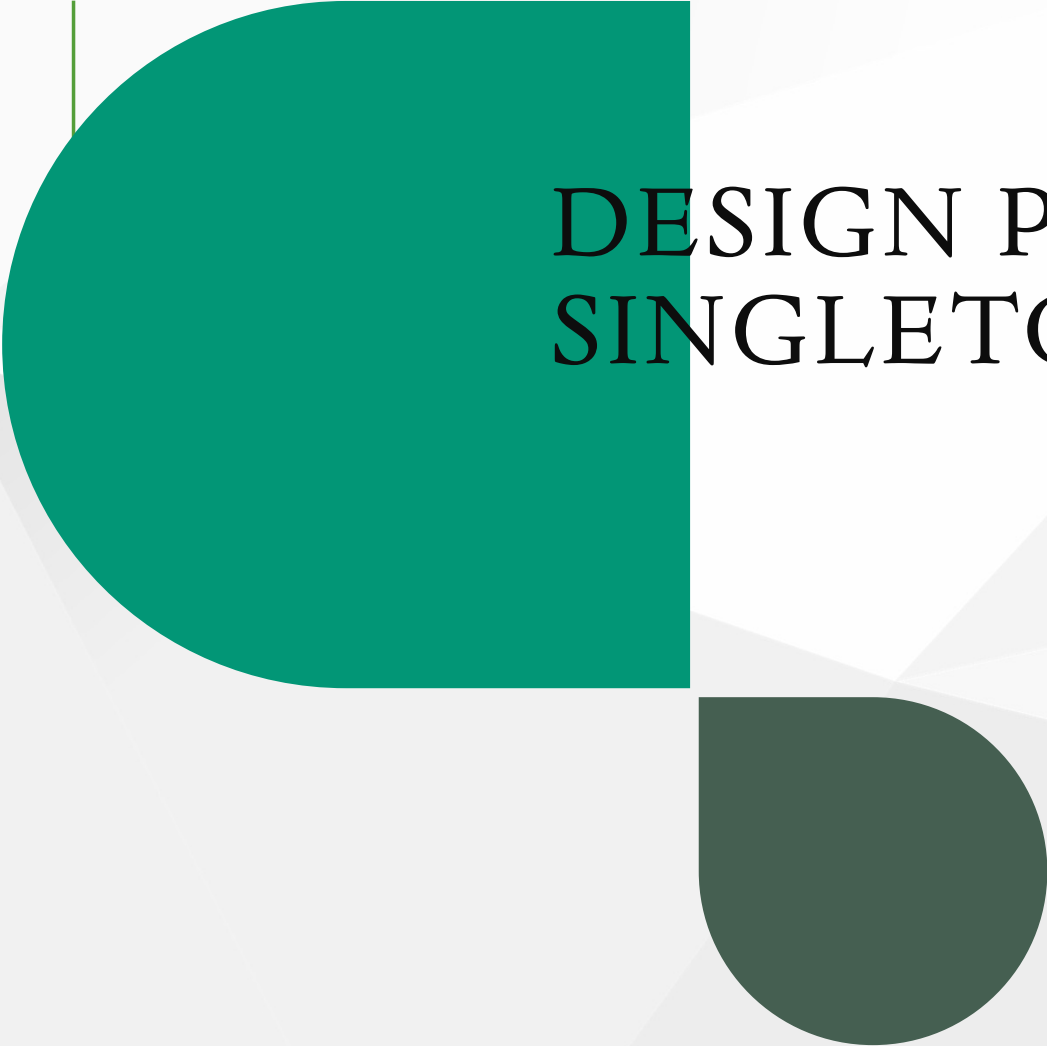
Isolation Level	Transactions	Dirty Reads	Non-Repeatable Reads	Phantom Reads
TRANSACTION_NONE	Not supported	<i>Not applicable</i>	<i>Not applicable</i>	<i>Not applicable</i>
TRANSACTION_READ_COMMITTED	Supported	Prevented	Allowed	Allowed
TRANSACTION_READ_UNCOMMITTED	Supported	Allowed	Allowed	Allowed
TRANSACTION_REPEATABLE_READ	Supported	Prevented	Prevented	Allowed
TRANSACTION_SERIALIZABLE	Supported	Prevented	Prevented	Prevented

- Habituellement, vous n'avez pas besoin de faire quoi que ce soit concernant le niveau d'isolation de la transaction.
- Vous pouvez simplement utiliser le niveau par défaut pour votre SGBD. Par exemple, pour Java, il est `TRANSACTION_READ_COMMITTED`.
- JDBC vous permet de savoir à quel niveau d'isolation de transaction votre SGBD est réglé (`getTransactionIsolation`) et vous permet également de le mettre à un autre niveau (`setTransactionIsolation`).

LES TRANSACTIONS : JALON

- `setSavePoint()` : Cette méthode vous permet de positionner un jalon dans votre transaction. Ce jalon peut alors être utilisé avec la méthode `rollback()`
- `releaseSavePoint()` : Cette méthode permet de supprimer un jalon de votre transaction.

```
1.  try{
2.      //définir la gestion des transactions manuelles
3.      conn.setAutoCommit(false);
4.      Statement stmt = conn.createStatement();
5.
6.      //définir un point de sauvegarde
7.      Savepoint savepoint = conn.setSavepoint("Savepoint1");
8.
9.      String query = "INSERT INTO Emp VALUES (101, 'Alex')";
10.     stmt.executeUpdate(query);
11.
12.     //Soumettre une instruction SQL mal formée qui brise le code
13.     String query = "INSERT IN Emp VAL (102, 'Bob')";
14.     stmt.executeUpdate(query);
15.
16.     // S'il n'y a pas d'erreur.
17.     conn.commit();
18. }
19. catch(SQLException se){
20.     // S'il y a une erreur.
21.     conn.rollback(savepoint);
22. }
```

DESIGN PATTERN SINGLETON

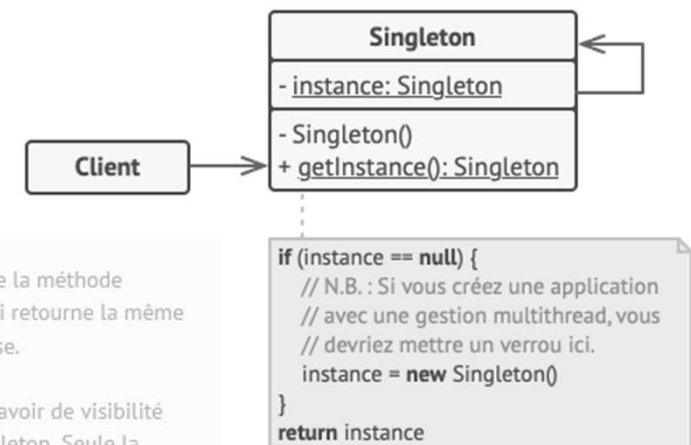
PATTERN SINGLETON

Singleton est un patron de conception de création qui garantit que **l'instance d'une classe n'existe qu'en un seul exemplaire**, tout en fournissant un point d'accès global à cette instance.

Le singleton règle deux problèmes à la fois, mais ne respecte pas le principe de responsabilité unique.

- Il garantit l'unicité d'une instance pour une classe.
- Il fournit un point d'accès global à cette instance.

Structure



SINGLETON

Le constructeur de notre singleton

```
public class Singleton { 8 usages

    private static final Properties props = new Properties(); 5 usages
    private static Connection connection; 4 usages

    private final String PATHCONF = "conf.properties"; 1 usage

    private Singleton() { 1 usage

        try(InputStream is = Singleton.class.getClassLoader().getResourceAsStream(PATHCONF)) {

            props.load(is);

        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }

        try {

            // chargement du driver
            Class.forName(props.getProperty("jdbc.driver.class"));

            // Création de la connection
            String url = props.getProperty("jdbc.url");
            String login = props.getProperty("jdbc.login");
            String password = props.getProperty("jdbc.password");
            connection = DriverManager.getConnection(url, login, password);

            // a retirer !!! juste informatif
            System.out.println("Connected to database : " + connection);

        } catch (ClassNotFoundException e) {
            // si driver introuvable
            System.err.printf("Error loading JDBC Driver: %s\n", e.getMessage());
        } catch (SQLException e) {
            // si connexion impossible
            System.err.printf("Error Connexion JDBC Driver: %s\n", e.getMessage());
        }

    }

}
```

SINGLETON

Les méthodes de gestion de l'instance :

- `getInstanceDB()` : retourne l'instance en cours sinon la crée
- `closeInstanceDB()` : ferme l'instance en cours
- `getConnection()` : retourne l'instance. Cette méthode est en private.

```
public static Connection getInstanceDB() { 4 usages

    try {
        if (getConnection() == null || getConnection().isClosed()) {

            new Singleton();
            System.out.println("Connected to database : " + getConnection());
        } else {
            System.out.println("Connection already existing");
        }
    } catch (SQLException e) {
        System.out.printf("Error Connexion_v1 JDBC Driver: %s\n", e.getMessage());
    }

    return connection;
}

public static void closeInstanceDB() { no usages
    try {
        if (getConnection() != null && !getConnection().isClosed()) {
            getConnection().close();
        }
    } catch (SQLException e) {
        System.out.printf("Error Closing JDBC Connection: %s\n", e.getMessage());
    }
}

> private static Connection getConnection() { return connection; }
}
```

TEST AVEC SINGLETON

Désormais , je fais appel à mon Singleton pour initialiser la connexion à la base de données.

```
/* ----- METHODE SINGLETON ----- */
private List<Personne> findAll() {...}

private Personne findById(int pId) { 1 usage ± neojero *

    Personne personne = new Personne();
    StringBuilder selectById = new StringBuilder("select * from personne where per_id=?");

    try {
        Connection con = Singleton.getInstanceDB();
        // contrôle
        System.out.println(con);

        // utilisation d'un preparedStatement
        PreparedStatement pstmt = con.prepareStatement(selectById.toString());

        pstmt.setInt( parameterIndex: 1, pId);

        ResultSet resultSet = pstmt.executeQuery();

        while(resultSet.next()) {

            personne.setId(resultSet.getInt( columnLabel: "per_id"));
            personne.setCivileId(resultSet.getInt( columnLabel: "civ_id"));
            personne.setNom(resultSet.getString( columnLabel: "per_nom"));
            personne.setPrenom(resultSet.getString( columnLabel: "per_prenom"));
            personne.setAdresse(resultSet.getString( columnLabel: "per_adr"));
            personne.setEmail(resultSet.getString( columnLabel: "per_cp"));
            personne.setVille(resultSet.getString( columnLabel: "per_ville"));
            personne.setTelephone(resultSet.getInt( columnLabel: "per_tel"));
            personne.setFax(resultSet.getInt( columnLabel: "per_fax"));
            personne.setEmail(resultSet.getString( columnLabel: "per_email"));
            personne.setDateNaissance(resultSet.getDate( columnLabel: "per_datenaissance"));

        }

        return personne;

    } catch (SQLException e) {
        throw new RuntimeException(e);
    }
}
```